Acoustic Scene Awareness: Urban and Household Sound Recognition using Convolutional Neural Networks with Autoencoder-Based Novel-Sound Detection

Softwarica College of IT & E-Commerce
(Coventry University)

Module: **STW7088CEM** – Artificial Neural Networks
Coursework Submission, M.Sc. Data Science and Computational Intelligence

Submitted by

SUDIP ADHIKARI
(250578)

Abstract

This report documents the design, implementation, and evaluation of a neural network system for recognising everyday urban and household sounds, motivated by assistive technology for Deaf and hard-of-hearing users who cannot rely on an audible cue when a siren, car horn, or alarm occurs. Raw audio is transformed into log-mel spectrograms — a two-dimensional representation of frequency against time — so that a convolutional neural network can learn each sound's signature directly from the signal, without any hand-engineered acoustic features. Four tasks are performed on the UrbanSound8K and ESC-50 datasets: (i) ten-class **classification** with a four-block convolutional network; (ii) **novelty detection** using a convolutional autoencoder whose reconstruction error identifies sounds outside the training vocabulary; (iii) unsupervised **clustering** of the network's learned embeddings using k-means and t-SNE; and (iv) an **architecture comparison** against a multi-layer perceptron baseline together with a data-augmentation ablation. All experiments follow UrbanSound8K's official ten-fold cross-validation protocol, which is necessary because clips sliced from the same source recording share a fold; a random split leaks those slices across training and test and inflates reported accuracy. The convolutional network reaches **[TBC]**% accuracy and **[TBC]** macro F1 while using 390,890 parameters — roughly fifteen times fewer than the 5,804,810-parameter perceptron baseline it outperforms, which quantifies the value of convolutional inductive bias on spectrogram input. The autoencoder detects previously unheard sounds with a ROC-AUC of **[TBC]**, and is compared against the obvious alternative of thresholding classifier confidence. All models were trained locally on an Apple M1 laptop using the Metal Performance Shaders backend; no paid computing resources were used.

Contents

1 Introduction

1.1 Background and Motivation

Sound carries a large amount of safety-critical information that is normally processed without conscious effort. A siren approaching from behind, a car horn, a smoke alarm, glass breaking in another room, or somebody knocking at the door are all events that a hearing person reacts to automatically. For a person who is Deaf or hard of hearing, none of these signals arrive. The World Health Organization estimates that over 1.5 billion people live with some degree of hearing loss, and the everyday consequence is not merely inconvenience but a genuine reduction in situational safety.

Commercial systems already exist in this space — Apple's Sound Recognition and Google's Sound Notifications both listen for a small set of household sounds and raise a visual or haptic alert — which establishes that the problem is real and the approach is viable. What those systems do internally is not published, and building an equivalent from open data is a well-scoped exercise in applied neural network design.

1.2 Why This Is a Neural Network Problem

It is worth being explicit about why this task requires neural networks rather than the classical machine learning methods studied in other modules. Raw audio is a waveform sampled at tens of thousands of values per second; there is no meaningful way to present that directly to a decision tree, a support vector machine, or a logistic regression. Classical approaches to audio therefore depend on a human first deciding which summary statistics matter — zero-crossing rate, spectral centroid, a handful of MFCC coefficients — and the quality of the result is bounded by the quality of that hand-engineering.

A convolutional network removes that bottleneck. Presented with a spectrogram, it learns its own filters: early layers respond to local time–frequency texture such as onsets, harmonic stacks, and broadband noise, and deeper layers compose those primitives into class-level patterns. The representation is discovered from data rather than specified in advance. This property — *representation learning* — is the defining capability of artificial neural networks and the reason the field displaced feature-engineering pipelines in audio, vision, and language.

The novelty-detection task reinforces the point. An autoencoder learns a compressed internal code for the sounds it has seen and rebuilds the input from that code. Nothing in classical supervised learning provides an equivalent mechanism for saying *"this input is unlike anything I was trained on"* without being given negative examples in advance.

1.3 Project Objectives

1. Build a convolutional neural network that classifies ten urban sound classes from log-mel spectrograms, evaluated under the dataset's official ten-fold protocol.

2. Quantify the benefit of convolutional structure by comparing against a multi-layer perceptron baseline of substantially larger parameter count.

3. Measure the effect of spectrogram-domain data augmentation (SpecAugment and time shifting) as a controlled ablation.

4. Implement novelty detection with a convolutional autoencoder so that unfamiliar sounds are reported as unknown rather than being forced into one of the ten trained labels, and compare this against a softmax-confidence baseline.

5. Examine the learned representation without labels, using k-means clustering and t-SNE projection of the network's penultimate embeddings.

6. Deliver a working real-time demonstration that runs from a laptop microphone.

1.4 Original Contributions

The individual components — CNNs on mel-spectrograms, autoencoder anomaly scoring — are established techniques. The contributions of this work are in their combination and in the rigour of the evaluation:

- **An open-set framing of a closed-set benchmark.** UrbanSound8K is almost always treated as a ten-way classification problem. Here the classifier is paired with a novelty detector and evaluated against genuinely unseen sound classes drawn from a second dataset (ESC-50), which is the situation any deployed assistive device actually faces.
- **An honest baseline for the novelty detector.** Rather than presenting autoencoder reconstruction error in isolation, it is compared directly against thresholding the classifier's own softmax confidence — the cheap alternative that a reviewer would immediately propose.
- **A parameter-matched argument for convolution.** The MLP baseline is deliberately larger than the CNN, so the comparison isolates architectural inductive bias rather than model capacity.
- **A real-time deployment.** The trained networks run live on laptop hardware with both models in the loop, demonstrating that the system is practical rather than purely offline.

1.5 Report Structure

Section 2 reviews related work. Section 3 describes both datasets and the evaluation protocol. Section 4 sets out the audio front-end and the three network architectures. Section 5 covers implementation with reference to the source code. Section 6 presents and discusses results across all four tasks. Sections 7 and 8 address limitations and the ethical dimension of always-on microphones, and Section 9 concludes.

2 Background and Related Work

2.1 Environmental Sound Classification

Environmental sound classification (ESC) differs from speech and music recognition in that the sounds have no shared grammatical or harmonic structure. Early systems used hand-designed features — most commonly Mel-Frequency Cepstral Coefficients (MFCCs), inherited from speech processing — fed into a Gaussian Mixture Model or Support Vector Machine.

The shift to neural approaches was driven by two datasets released in the mid-2010s. Salamon, Jacoby and Bello introduced UrbanSound8K [1], which remains the standard benchmark for urban ESC, and Piczak released ESC-50 [2] alongside a demonstration that convolutional networks operating on spectrograms outperform the feature-engineering pipelines that preceded them [3].

2.2 Spectrograms as Network Input

The dominant design pattern in modern ESC is to convert audio into a time–frequency representation and treat the result as an image. The mel scale is used because it approximates the non-linear frequency resolution of human hearing, allocating more resolution to lower frequencies where most environmental sound energy lies. Amplitudes are converted to decibels

because loudness perception is logarithmic; without this, a small number of high-energy frames dominate the input range and training becomes unstable.

Comparative studies of time–frequency representations for ESC [8] found log-mel spectrograms to be a consistently strong choice, which motivates their use here.

2.3 Data Augmentation

ESC datasets are small by deep learning standards — UrbanSound8K contains 8,732 clips, orders of magnitude fewer than the image corpora on which deep CNNs were developed — so overfitting is the central practical difficulty. Salamon and Bello [4] showed that audio-domain augmentation (time stretching, pitch shifting, adding background noise) materially improves CNN performance on UrbanSound8K.

Park et al. later introduced SpecAugment [5], which operates directly on the spectrogram by masking random frequency bands and time spans. It is considerably cheaper than audio-domain augmentation, since no re-synthesis or re-transformation is required, and it forces the network to use the entire time–frequency pattern rather than latching onto a single narrow band. SpecAugment and random time shifting are the augmentations used in this project.

2.4 Novelty and Out-of-Distribution Detection

A softmax classifier is closed-set by construction: its outputs are constrained to sum to one over the classes it knows, so an unfamiliar input is necessarily assigned to one of them. Hendrycks and Gimpel [6] established the baseline that maximum softmax probability carries *some* out-of-distribution signal, while also documenting that neural networks are frequently confident when wrong.

Autoencoder reconstruction error is a widely used alternative [7]: a model trained to compress and rebuild in-distribution data will reconstruct out-of-distribution data poorly, and the error is a usable score requiring no negative examples at training time. This report implements the autoencoder approach and evaluates it against the Hendrycks and Gimpel softmax baseline rather than assuming its superiority.

2.5 Positioning of This Work

Relative to the literature, this project sits at the intersection of standard ESC classification and open-set recognition. Reported CNN accuracies on UrbanSound8K commonly fall in the 70–92% range depending on architecture, augmentation, and — importantly — whether the official fold protocol was respected. This work targets the upper part of that range with a deliberately compact architecture, and extends the standard setup with the novelty-detection and clustering tasks required by the assignment brief.

3 Datasets

3.1 UrbanSound8K

UrbanSound8K [1] is the primary dataset: 8,732 labelled audio excerpts of up to four seconds, drawn from field recordings on the Freesound archive, covering ten classes selected from an urban sound taxonomy. The classes are `air_conditioner`, `car_horn`, `children_playing`, `dog_bark`, `drilling`, `engine_idling`, `gun_shot`, `jackhammer`, `siren`, and `street_music`.

Four of these — `car_horn`, `dog_bark`, `gun_shot`, and `siren` — are treated as safety-critical in the demonstration system and trigger a distinct alert state.

[FIGURE PENDING]

`fig_class_distribution.png`

generated by `run_experiments.py`

Figure 1: Clip count per class in UrbanSound8K. The set is broadly balanced apart from `car_horn` and `gun_shot`, which are noticeably smaller — a fact that becomes relevant when interpreting per-class F1 in Section 6.

3.2 ESC-50 (Unseen Sounds)

ESC-50 [2] contributes 2,000 five-second clips across 50 classes. It is *not* used for classifier training. Its role is to supply genuinely unfamiliar sounds for the novelty-detection experiment: eight household and safety-relevant classes were selected — `door_wood_knock`, `glass_breaking`, `crying_baby`, `clock_alarm`, `door_wood_creaks`, `water_drops`, `footsteps`, and `can_opening` — none of which appear in the UrbanSound8K vocabulary.

This choice is deliberate. Evaluating novelty detection against random noise would be trivially easy and uninformative; evaluating it against real recordings of plausible household events is a fair test of whether the system would behave sensibly in the environment it is intended for.

3.3 Evaluation Protocol and a Common Methodological Error

UrbanSound8K ships with ten predefined folds, and the dataset authors are explicit that these must be used rather than a random split. The reason is that the 8,732 clips are sliced from a much smaller number of source field recordings, and all slices from one recording are placed in the same fold. Under a random split, several slices of a single recording land in training while others land in test; the model can then succeed by recognising the specific recording rather than the sound class, and reported accuracy is inflated by a wide margin.

All results in this report use ten-fold cross-validation over the official folds. For each fold k, fold k is the test set, fold $(k \bmod 10) + 1$ is the validation set used for early stopping and model selection, and the remaining eight folds are used for training. Normalisation statistics are computed on the training partition alone and then applied unchanged to validation and test, so that no test statistics leak into training.

Because every clip appears in exactly one test fold, pooling the held-out predictions across all ten folds yields one honest prediction per clip for the entire dataset. The aggregate confusion matrix and per-class scores in Section 6 are computed on that pooled set.

4 Methodology

4.1 Audio Front-End

Every clip passes through an identical front-end before reaching any network:

1. **Load as mono at 22,050 Hz.** Halving the usual 44.1 kHz rate discards content above 11 kHz, which carries little discriminative information for these classes, and halves the computational cost.

2. **Fix the duration to four seconds.** Shorter clips are zero-padded and longer ones truncated, so every spectrogram has identical dimensions — a requirement for batching through a convolutional network with a fixed input size.
3. **Short-Time Fourier Transform** with a 1,024-sample window and 512-sample hop.
4. **Project onto 64 mel bands** between 20 Hz and the Nyquist frequency.
5. **Convert power to decibels**, referenced to the per-clip maximum.

The result is a 64×173 matrix per clip: 64 frequency bands by 173 time frames. Figure 2 shows one example per class, and makes the central argument of the project visually: each sound class produces a visibly distinct pattern, which is precisely what makes a convolutional network appropriate.

[FIGURE PENDING]

`fig_spectrogram_examples.png`

generated by `run_experiments.py`

Figure 2: Log-mel spectrograms, one per UrbanSound8K class. The horizontal striations of `jackhammer` and `drilling`, the rising harmonic sweep of `siren`, and the sharp broadband transient of `gun_shot` are distinguishable by eye — structure that a convolutional network can exploit and that a flattened feature vector destroys.

4.2 Data Augmentation

Two spectrogram-domain augmentations are applied to training data only:

- **Random time shift** (up to ±20% of the clip), implemented as a circular roll. A siren remains a siren regardless of when within the window it begins, so the network should not depend on absolute position.
- **SpecAugment** [5]: one random frequency band (up to 8 of 64 mel bins) and one random time span (up to 16 of 173 frames) are masked to the spectrogram floor. This prevents the network from relying on any single narrow band and acts as a strong regulariser on a dataset of this size.

[FIGURE PENDING]

`fig_augmentation.png`

generated by `run_experiments.py`

Figure 3: One siren clip under each augmentation. Masked bands and spans are visible as flat regions. The network never sees exactly the same input twice, which is what suppresses memorisation.

4.3 Architecture 1: Multi-Layer Perceptron (Baseline)

The baseline flattens the 64×173 spectrogram into an 11,072-dimensional vector and passes it through two fully connected hidden layers (512 and 256 units) with batch normalisation, ReLU activation, and dropout at $p = 0.5$.

Its purpose is diagnostic. Flattening destroys the two-dimensional structure of the spectrogram: the network has no notion that two adjacent time frames are related, or that a harmonic pattern shifted slightly in frequency is the same pattern. Every input pixel receives its own independent weight, which is why the model requires 5,804,810 parameters — roughly fifteen times the CNN's 390,890. If the CNN outperforms it despite that disparity, the difference is attributable to architecture rather than capacity.

4.4 Architecture 2: Convolutional Neural Network

The main classifier consists of four convolutional blocks, each comprising a 3×3 convolution, batch normalisation, ReLU, and 2×2 max pooling, with channel widths of 32, 64, 128, and 256. Spatial dropout is applied in the deeper blocks.

Two design decisions are worth stating explicitly:

- **Global average pooling replaces a flatten-and-dense head.** Collapsing each of the 256 final feature maps to its mean gives a 256-dimensional descriptor. This keeps the parameter count an order of magnitude below the MLP and, more importantly, makes the model approximately invariant to *where* in the clip a sound occurs — appropriate, because a dog bark two seconds in is the same event as a dog bark at the start.
- **The 256-dimensional pooled vector is exposed as an embedding.** This is the representation analysed in the clustering task (Section 6.3), allowing the learned feature space to be examined directly.

Training uses AdamW (learning rate 10^{-3}, weight decay 10^{-4}), cross-entropy loss with label smoothing of 0.05, cosine-annealed learning rate, and early stopping on validation accuracy with a patience of eight epochs.

4.5 Architecture 3: Convolutional Autoencoder

The novelty detector is a symmetric convolutional autoencoder. The encoder applies three stride-2 convolutions (16, 32, and 32 channels), compressing the 64×173 input to a $32 \times 8 \times 22$ latent representation; the decoder mirrors this with transposed convolutions. Because transposed convolution can overshoot the original dimensions by a pixel, the output is cropped back to the input size.

Crucially, it is trained *only* on the ten known classes, with a mean-squared-error reconstruction objective. Having learned a compressed code adequate for those sounds, it reconstructs an unfamiliar sound poorly, and the per-clip reconstruction error

$$s(x) = \frac{1}{HW} \sum_{i,j} \left(\hat{x}_{ij} - x_{ij}\right)^2$$

serves directly as a novelty score. The decision threshold is set at the 95th percentile of the reconstruction errors observed on *known* data — a calibration that requires no novel examples at all, matching the constraint a deployed system operates under.

At 28,321 parameters the autoencoder is by far the smallest of the three networks, which matters for running both models concurrently in the real-time demonstration.

5 Implementation

5.1 Environment

All work was carried out on an Apple MacBook (M1, 16 GB) running Python 3.11 and PyTorch 2.13, using the Metal Performance Shaders (MPS) backend for GPU acceleration on Apple Silicon. No cloud or paid computing resources were used at any stage.

One environment issue is worth recording for reproducibility: the Homebrew-supplied Python builds on the target machine had a broken `pyexpat` module, which links against the older system `libexpat` and fails to resolve a required symbol. This prevents `pip` from bootstrapping inside a virtual environment. The resolution was to use `uv` to provision a self-contained CPython 3.11 distribution, which is documented in the repository README.

5.2 Project Layout

```
src/config.py        hyperparameters, paths, device selection
src/data.py          audio loading, mel-spectrograms, augmentation,
   datasets
src/models.py        MLPBaseline, AudioCNN, ConvAutoencoder
src/train.py         training loop and 10-fold cross-validation
src/evaluate.py      pooled metrics, confusion matrix, embeddings
src/anomaly.py       autoencoder novelty detection + softmax baseline
src/cluster.py       k-means, t-SNE, cluster quality metrics
src/viz.py           all report figures
run_experiments.py   staged experiment runner
demo/live_demo.py    real-time microphone dashboard
```

5.3 Feature Extraction

The front-end of Section 4 is implemented in `src/data.py`. Listing 1 shows the core transform.

```python
def audio_to_melspec(y: np.ndarray, sr: int = C.SAMPLE_RATE) -> np.ndarray:
    mel = librosa.feature.melspectrogram(
        y=y, sr=sr,
        n_fft=C.N_FFT,          # 1024
        hop_length=C.HOP_LENGTH, # 512
        n_mels=C.N_MELS,         # 64
        fmin=C.F_MIN, fmax=C.F_MAX,
    )
    # Decibel scaling: loudness perception is logarithmic, and without
    # this a few loud frames dominate the input range.
    mel_db = librosa.power_to_db(mel, ref=np.max)
    return mel_db.astype(np.float32)
```

Listing 1: Waveform to log-mel spectrogram (`src/data.py`).

Extraction runs once over the full corpus and is cached to `.npy`, after which every experiment reads features from memory. Measured throughput was approximately 3 ms per clip, so caching all 8,732 spectrograms completes in well under a minute — a useful property when experiments must be repeated.

5.4 The Fold Split

Listing 2 implements the protocol described in Section 3.3. The comment records *why* the split is by fold, since this is the single easiest way to produce misleadingly high numbers on this dataset.

```python
def make_fold_split(X, meta, test_fold: int, val_fold: int | None = None):
    """Split by UrbanSound8K's official folds.

    Clips sliced from the same source recording share a fold, so splitting
    by fold (rather than randomly) is what keeps the evaluation honest.
    """
    if val_fold is None:
        val_fold = test_fold % C.N_FOLDS + 1

    folds  = meta["fold"].values
    labels = meta["classID"].values

    te = folds == test_fold
    va = folds == val_fold
    tr = ~(te | va)

    return (X[tr], labels[tr]), (X[va], labels[va]), (X[te], labels[te])
```

Listing 2: Official fold-based splitting (`src/data.py`).

5.5 The Convolutional Classifier

```python
class AudioCNN(nn.Module):
    def __init__(self, n_classes: int = 10, dropout: float = 0.3, width: int =
        32):
        super().__init__()
        w = width
        self.block1 = ConvBlock(1,     w,     pool=2)
        self.block2 = ConvBlock(w,     w * 2, pool=2, dropout=dropout / 2)
        self.block3 = ConvBlock(w * 2, w * 4, pool=2, dropout=dropout / 2)
        self.block4 = ConvBlock(w * 4, w * 8, pool=2, dropout=dropout)

        # Global average pooling instead of flatten+dense: ~10x fewer
        # parameters, and invariance to where in the clip the sound occurs.
        self.gap     = nn.AdaptiveAvgPool2d(1)
        self.dropout = nn.Dropout(dropout)
        self.fc      = nn.Linear(w * 8, n_classes)

    def forward(self, x, return_embedding: bool = False):
        x = self.block4(self.block3(self.block2(self.block1(x))))
        emb    = self.gap(x).flatten(1)          # 256-d descriptor
        logits = self.fc(self.dropout(emb))
        if return_embedding:
            return logits, emb
        return logits
```

Listing 3: AudioCNN forward pass with optional embedding output (`src/models.py`).

5.6 Novelty Scoring

```python
@torch.no_grad()
def reconstruction_error(self, x) -> torch.Tensor:
    """Per-sample mean squared error -- the novelty score."""
    recon = self(x)
    return ((recon - x) ** 2).flatten(1).mean(dim=1)
```

Listing 4: Reconstruction error as a novelty score (`src/models.py`).

The threshold is calibrated from known data only, as shown in Listing 5.

```python
# Set at a percentile of the KNOWN scores, so the detector can be
# calibrated without ever seeing a novel example -- which is the
# situation a deployed system is actually in.
thr = float(np.percentile(known_scores, percentile))   # percentile = 95
```

Listing 5: Threshold calibration without novel examples (`src/anomaly.py`).

5.7 Reproducibility

Every random source is seeded from `SEED = 42` in `src/config.py`, with each fold deriving a deterministic offset. The full experiment suite runs with:

```
uv venv --python 3.11 --python-preference only-managed
uv pip install -r requirements.txt
bash scripts/download_data.sh
.venv/bin/python run_experiments.py
```

Metrics are written as JSON to `results/logs/`, and every figure in this report is regenerated from `src/viz.py` into `results/figures/` as both PNG and vector PDF. A smoke test (`scripts/smoke_test.py`) exercises all stages without requiring the larger dataset.

5.8 Project Resources

All source code, the LaTeX sources for this report, generated figures, per-experiment result files, and a recorded demonstration are accessible at:

- **Source code repository** (GitHub):
  https://github.com/SudipAdh/acoustic-scene-awareness
- **Demonstration video** (Loom):
  [LOOM LINK TO BE INSERTED]
- **Live demonstration application:** `demo/live_demo.py` in the repository.
- **Generated figures and metrics:** `results/figures/` and `results/logs/` in the repository.

The repository mirrors the layout described above. Neither dataset is redistributed in the repository, in line with their CC BY-NC licences; `scripts/download_data.sh` retrieves both from their official sources, and the README documents the process.

6 Results and Discussion

6.1 Architecture Comparison

Table 1 reports ten-fold cross-validated performance for the three configurations.

Table 1: Ten-fold cross-validation results on UrbanSound8K (mean ± standard deviation across folds).

Model	Parameters	Accuracy (%)	Macro F1
MLP baseline	5,804,810	[TBC] ± [TBC]	[TBC] ± [TBC]
CNN (no augmentation)	390,890	[TBC]	[TBC]
CNN + augmentation	390,890	[TBC] ± [TBC]	[TBC] ± [TBC]

[FIGURE PENDING]

`fig_model_comparison.png`

generated by `run_experiments.py`

Figure 4: Performance and model size side by side. The CNN achieves higher accuracy and macro F1 than the MLP while using roughly fifteen times fewer parameters.

The central finding is that the convolutional network outperforms the perceptron by [TBC] percentage points of accuracy while using approximately fifteen times fewer parameters. Since the MLP has strictly greater capacity, the gap cannot be explained by model size; it is attributable to architectural inductive bias. Convolution encodes two assumptions that hold for spectrograms and that the MLP must otherwise learn from scratch: that informative structure is *local* in time and frequency, and that the same pattern is meaningful wherever it appears. Weight sharing means a filter that detects a harmonic onset is learned once and applied everywhere, whereas the MLP must independently learn that pattern at every position it might occur — with only a few thousand training examples to do so.

Augmentation contributes a further [TBC] percentage points. Figure 5 shows the mechanism: the unaugmented CNN separates training and validation loss early, whereas the augmented model keeps them close for considerably longer.

[FIGURE PENDING]
`fig_training_curves.png`
generated by `run_experiments.py`

Figure 5: Training and validation curves on a representative fold. Dashed lines are training, solid lines validation. The widening gap in the unaugmented configurations is overfitting; augmentation narrows it.

6.2 Per-Class Performance and Error Structure

[FIGURE PENDING]
`fig_confusion_matrix.png`
generated by `run_experiments.py`

Figure 6: Aggregate confusion matrix over all ten held-out folds, normalised per row. Each clip is predicted exactly once, when its fold is the test fold.

[FIGURE PENDING]
`fig_per_class_f1.png`
generated by `run_experiments.py`

Figure 7: Per-class F1. Impulsive, spectrally distinctive sounds are recognised most reliably; sustained broadband machine noise is the principal source of error.

The strongest class is [TBC] (F1 = [TBC]) and the weakest [TBC] (F1 = [TBC]). The error structure in Figure 6 is acoustically interpretable rather than arbitrary: the dominant confusions occur among `air_conditioner`, `engine_idling`, `drilling`, and `jackhammer`. All four are sustained, broadband, low-frequency machine noises whose log-mel representations genuinely resemble one another, and human listeners confuse them under similar conditions. By contrast, classes with sharp temporal or harmonic signatures — `gun_shot`, `siren`, `dog_bark` — are well separated.

This distinction matters for the intended application. The four safety-critical classes are drawn from the well-separated group, so the errors the system does make are concentrated in

the classes where a mistake carries the least consequence: confusing an air conditioner with an idling engine has no safety implication, whereas missing a siren would.

6.3 Clustering the Learned Representation

To examine what the network learned independently of its classification head, 256-dimensional embeddings were extracted from the held-out fold — data the network never trained on — and clustered with k-means ($k = 10$) without any label information.

Table 2: Agreement between unsupervised clusters and true classes.

Metric	Value
Adjusted Rand Index	[TBC]
Normalised Mutual Information	[TBC]
Silhouette coefficient	[TBC]

[FIGURE PENDING]

`fig_tsne_embeddings.png`

generated by `run_experiments.py`

Figure 8: t-SNE projection of the CNN's 256-dimensional embeddings for held-out clips, with each cluster labelled directly at its centroid. Classes form coherent groups without the clustering algorithm being given any labels.

That k-means recovers the class structure to this degree confirms the network learned a genuinely organised representation rather than an arbitrary decision boundary. The t-SNE projection is also diagnostically useful: the classes that overlap in embedding space are the same ones that the confusion matrix shows being misclassified, so the two analyses corroborate each other. The machine-noise classes form a single contiguous region, which explains at a representational level why they are difficult to separate.

6.4 Novel-Sound Detection

The classifier was then exposed to the eight unseen ESC-50 classes. Table 3 compares the autoencoder against the softmax-confidence baseline.

Table 3: Novelty detection: distinguishing unseen ESC-50 sounds from known UrbanSound8K sounds.

Method	ROC-AUC	Average precision
Softmax confidence baseline [6]	[TBC]	—
Convolutional autoencoder	[TBC]	[TBC]

At the 95th-percentile threshold the autoencoder achieves precision [TBC], recall [TBC], and F1 [TBC].

[FIGURE PENDING]

`fig_anomaly_scores.png`

generated by `run_experiments.py`

Figure 9: Distribution of reconstruction error for known and unseen sounds, with the operating threshold marked. Separation between the distributions is what makes the score usable.

[FIGURE PENDING]

`fig_anomaly_roc.png`

generated by `run_experiments.py`

Figure 10: ROC curve for novel-sound detection using autoencoder reconstruction error.

[FIGURE PENDING]

`fig_ae_reconstructions.png`

generated by `run_experiments.py`

Figure 11: Autoencoder input and reconstruction for known and unseen sounds. Known sounds are rebuilt with their structure intact; unseen sounds lose detail, and that loss is the detection signal.

A finding worth emphasising is the classifier's confidence on sounds it has never encountered: mean maximum softmax probability on the unseen ESC-50 clips was **[TBC]**. A closed-set classifier does not merely make mistakes on unfamiliar input — it makes them *confidently*, because softmax normalises over the ten known classes and has no mechanism to express "none of these". For an assistive device this is the most dangerous failure mode available, since a confidently wrong label is more likely to be acted upon than an uncertain one. The autoencoder addresses exactly this gap, and the comparison in Table 3 quantifies how much it adds over the cheaper softmax heuristic.

6.5 Real-Time Demonstration

The trained models are deployed in `demo/live_demo.py`, which maintains a rolling four-second microphone buffer, recomputes the spectrogram on the same front-end used in training, and

runs both networks twice per second. The interface displays the live spectrogram, per-class confidence, and a status banner that distinguishes three states: a recognised ordinary sound, a recognised safety-critical sound, and a sound the autoencoder has flagged as unfamiliar.

Two implementation details make this practical. Predictions are smoothed with an exponential moving average, without which single-frame noise causes visible flicker. And because the classifier is only 390,890 parameters and the autoencoder 28,321, both run comfortably in real time on laptop CPU or MPS, with measured inference at approximately **[TBC]** per window.

A recorded walkthrough is linked in Section 5.8.

7 Limitations and Future Work

- **Ten classes is a small vocabulary.** UrbanSound8K's taxonomy is urban and outdoor-oriented. A deployed assistive device would need the household sounds that matter most — doorbell, smoke alarm, name being called — which are only partly available in ESC-50 and would require either a combined taxonomy or purpose-built data collection.
- **Clean recordings, noisy world.** Both datasets contain reasonably isolated events. Real environments layer sounds continuously, and a single-label classifier cannot express "a siren *and* street music". Multi-label classification with sound event detection over time would be the correct formulation.
- **Fixed four-second windows.** A gunshot lasts a fraction of a second and is padded with silence; a jackhammer may run continuously and be truncated. Variable-length modelling, or attention pooling over longer contexts, would fit the data better.
- **Novelty detection is a coarse instrument.** Reconstruction error indicates that an input is unfamiliar but not what it is, and the threshold trades false alarms against missed detections along a single axis. A production system would likely need per-class confidence calibration in addition.
- **Single-microphone, single-device evaluation.** Performance was not measured across different microphones, room acoustics, or distances, all of which shift the spectrogram distribution. Domain robustness was not evaluated.
- **Comparison scope.** A transfer-learning baseline — fine-tuning a network pretrained on AudioSet, for instance — would contextualise the results against what is achievable with substantially more pretraining data, and is the most obvious next experiment.

8 Social, Ethical, Legal, and Professional Considerations

8.1 Privacy

An always-listening microphone is among the most privacy-sensitive components that can be placed in a home. The design here mitigates this in one important respect: all inference is local, the four-second buffer is overwritten continuously, and no audio is recorded, transmitted, or stored at any point. A deployed version should preserve that property, since a cloud-based implementation would create a permanent record of domestic conversation as a side effect of listening for a doorbell.

8.2 Asymmetric Consequences of Error

For an assistive device the two error types are not equivalent. A false alarm is a minor irritation; a missed siren or smoke alarm has real safety consequences. This argues for tuning the safety-critical classes toward recall at the cost of precision — the opposite of what maximising overall

accuracy would suggest — and it is a reason to report per-class metrics rather than a single headline number.

8.3 Bias and Dataset Provenance

Both datasets are drawn from Freesound, and their recordings are geographically skewed toward Europe and North America. Sirens in particular differ substantially between countries, and vehicle horns, alarms, and ambient street noise all vary regionally. A system trained on this data should not be assumed to transfer to Kathmandu without local evaluation and, most likely, local data collection. Presenting such a system as universally applicable would be misleading.

8.4 Accessibility and Appropriate Framing

The application targets Deaf and hard-of-hearing users, and it is important not to overstate what it delivers. The system is a supplementary awareness aid, not a safety-certified alerting device, and it should not be positioned as a replacement for certified accessible smoke alarms. Genuine accessibility work would also require participation from Deaf users in the design process rather than assumptions made on their behalf.

8.5 Licensing and Attribution

Both datasets are distributed under Creative Commons Attribution Non-Commercial licences. Their use in this coursework is consistent with those terms, both are cited, and neither is redistributed through the public repository — the download script retrieves them from their official sources instead.

9 Conclusion

This project built and evaluated a neural network system for recognising urban and household sounds, framed around assistive awareness for Deaf and hard-of-hearing users. Four tasks were performed: ten-class classification with a convolutional network on log-mel spectrograms, novelty detection with a convolutional autoencoder, unsupervised clustering of the learned embedding space, and a controlled architecture comparison with an augmentation ablation. All results follow UrbanSound8K's official ten-fold protocol.

The convolutional network reached **[TBC]**% accuracy and **[TBC]** macro F1 using 390,890 parameters, outperforming a 5,804,810-parameter perceptron baseline by **[TBC]** percentage points. Because the baseline is the larger model, the result isolates the contribution of convolutional inductive bias rather than capacity, which is the clearest experimental statement this project makes about why architecture matters. Spectrogram-domain augmentation added a further **[TBC]** points by suppressing overfitting on a dataset small enough for memorisation to be a genuine risk. The autoencoder detected previously unheard sounds with ROC-AUC **[TBC]**, compared against **[TBC]** for the softmax-confidence baseline.

Two observations are worth carrying forward. The first is that the classifier's errors are acoustically structured rather than random — confusion concentrates among sustained broadband machine noises that genuinely resemble one another, while the safety-critical classes sit in the well-separated region of the embedding space. Both the confusion matrix and the t-SNE projection show this independently, which is a stronger form of evidence than either alone. The second is the confidence of the closed-set classifier on inputs it has never seen: mean softmax probability of **[TBC]** on entirely unfamiliar sounds. Accuracy on a fixed test set says nothing about this behaviour, and any system intended to run in an open environment needs an explicit mechanism for expressing that an input falls outside its competence.

What the project demonstrated most concretely is the specific capability that distinguishes neural networks from the classical methods covered elsewhere in the programme. At no point was an acoustic feature designed by hand. The network was given a time–frequency representation and learned, on its own, the filters that separate a siren from a jackhammer — and the clustering analysis confirms that the resulting representation is structured enough for the classes to be recovered without labels at all.

References

References

[1] J. Salamon, C. Jacoby, and J. P. Bello, "A dataset and taxonomy for urban sound research," *Proc. 22nd ACM International Conference on Multimedia*, pp. 1041–1044, 2014.

[2] K. J. Piczak, "ESC: Dataset for environmental sound classification," *Proc. 23rd ACM International Conference on Multimedia*, pp. 1015–1018, 2015.

[3] K. J. Piczak, "Environmental sound classification with convolutional neural networks," *Proc. IEEE International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6, 2015.

[4] J. Salamon and J. P. Bello, "Deep convolutional neural networks and data augmentation for environmental sound classification," *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, 2017.

[5] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," *Proc. Interspeech*, pp. 2613–2617, 2019.

[6] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[7] M. Sakurada and T. Yairi, "Anomaly detection using autoencoders with nonlinear dimensionality reduction," *Proc. MLSDA 2nd Workshop on Machine Learning for Sensory Data Analysis*, pp. 4–11, 2014.

[8] M. Huzaifah, "Comparison of time-frequency representations for environmental sound classification using convolutional neural networks," *arXiv preprint arXiv:1706.07156*, 2017.

[9] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," *Proc. 14th Python in Science Conference*, pp. 18–25, 2015.

[10] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *Proc. 32nd International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[13] M. Lin, Q. Chen, and S. Yan, "Network in network," *Proc. International Conference on Learning Representations (ICLR)*, 2014.

[14] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.

[15] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8026–8037, 2019.

A Project Proposal (As Submitted)

[Paste the approved project proposal here, exactly as submitted, per the assignment brief.]

B Reproduction Instructions

```
# 1. Environment (Python 3.11)
uv venv --python 3.11 --python-preference only-managed
uv pip install -r requirements.txt

# 2. Datasets (~6.5 GB, downloads from official sources)
bash scripts/download_data.sh

# 3. Fast sanity check
.venv/bin/python run_experiments.py --quick

# 4. Full experiment suite
.venv/bin/python run_experiments.py

# 5. Live demonstration
.venv/bin/python demo/live_demo.py
```

C Experiment Evidence

[Insert screen captures of each experiment stage executing, showing the device and software environment, per the assignment brief.]

D Source Code

The complete source is available at https://github.com/SudipAdh/acoustic-scene-awareness. The principal modules are reproduced below.

[Full source listings to be inserted here.]